

Assignment 02 – DSA II Lab

Total Marks: 15 + 10(Viva)

Strict Deadline: June 6, 2026 (11:59 pm)

Strict Academic Integrity Warning

You are strictly prohibited from copying code from classmates, the internet, or any other external sources. You are strictly prohibited from using any generative AI tools to write, refactor, or structure your code. Furthermore, the viva is specifically designed to verify your authorship. If you cannot confidently explain the code you submitted, it will be treated as an academic integrity violation, resulting in an automatic zero for the assignment and further disciplinary action as per university policy.

Problem 1. Desert Expedition

A team of scientists must travel across a vast desert in a research rover to reach a remote **base camp**. The rover has a fuel tank with a maximum capacity of **K liters**. The rover starts at the **headquarters** with a **full tank**.

The desert region has **N stations** connected by **M trails**. Each trail has a fuel consumption cost — the rover **consumes that many liters** to travel that trail. If the rover does not have enough fuel to travel a trail, it **cannot take that route**.

Some stations are equipped with **fuel depots**. Whenever the rover arrives at a fuel depot station, it **automatically refuels to a full tank (K liters)**, regardless of how much fuel it had on arrival.

Your task is to find the **minimum fuel consumed** to travel from the **headquarters (source)** to the **base camp (destination)**. If it is impossible to reach the destination, print **-1**.

Input Format

N M K

src dst

F

f1 f2 f3 ... fF

u1 v1 w1

u2 v2 w2

...

uM vM wM

- N — number of stations ($2 \leq N \leq 1000$)
- M — number of trails ($1 \leq M \leq 10000$)
- K — maximum fuel tank capacity ($1 \leq K \leq 100$)
- src — starting station (headquarters)
- dst — destination station (base camp)
- F — number of fuel depot stations
- f1...fF — node indices of fuel depot stations

- Each of the next M lines: bidirectional trail between station u and v consuming w liters of fuel

Output Format

Minimum fuel consumed to reach base camp

If it is impossible to reach the destination, print:

-1

Constraints

$1 \leq N \leq 1000$

$1 \leq M \leq 10000$

$1 \leq K \leq 100$

$1 \leq w \leq K$

$\text{src} \neq \text{dst}$

All nodes are 1-indexed

Example 1

Input:

6 7 10

1 6

1

3

1 2 4

2 3 6

3 4 3

4 5 4

5 6 5

1 4 9

3 6 8

Output:

17

Example 2

Input:

4 4 6

1 4

1

2

1 2 3

2 3 3

3 4 3

1 3 5

Output:

9

Example 3 — Impossible Case

Input:

5 4 5
1 5
0

1 2 3
2 3 3
3 4 3
4 5 3

Output:

-1

Explanation:

- Tank K=5, no fuel depots.
- 1→2 (3, fuel: 5→2), 2→3 needs 3 but only 2 left — **stuck**.
- No other path exists. Impossible to reach station 5.

Hint: Standard Dijkstra tracks *distance[node]* — the minimum cost to reach a node. That is not enough here. A rover arriving at the same station with **different fuel levels** are in **completely different situations** — one may continue, the other may not. So, you have to expand the dimension of *distance* vector to track both **node index and current fuel level**.

Problem 2. (Will be uploaded soon)**Submission Guidelines**

Please follow these instructions exactly to ensure your assignment is graded properly. Incorrectly formatted submissions may receive a penalty or be rejected.

1. **Create a Folder:** Place your two completed C++ source files (e.g., Problem1.cpp and Problem2.cpp) into a single new folder.
2. **Rename the Folder:** Rename this folder to exactly match your University Student ID.
3. **Compress the Folder:** Compress the renamed folder into a .zip archive.
4. **Submit:** Upload and submit this single .zip file to the ELMS portal before the deadline.

Viva Evaluation (10 Marks)

Please note that submitting the code is only the first part of this assignment. There will be a **mandatory viva (oral examination)** based entirely on your submitted code, which will bear **10 marks** of your overall assignment grade.

During the viva, you will be asked to:

- Explain the step-by-step logic of your implementations.
- Walk through the time and space complexity of specific blocks of your code.